

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
MINGGU 5 PERTEMUAN 1
SEMESTER III



BKPM ACARA 9

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 7 dan 8, class Mahasiswa dirancang sebagai Model statis: seluruh method (all, find, create, update, delete) dipanggil langsung lewat nama class, dan Controller memanggil Database::getInstance() secara langsung di dalam Model. Pola ini praktis, tetapi menyulitkan pengujian, karena Controller dan Model saling terikat erat (tightly coupled) dengan satu implementasi koneksi database tertentu.

Acara ini menata ulang bagian Mahasiswa menggunakan tiga prinsip OOP lanjutan: Object Composition (Controller memiliki Repository, Repository memiliki koneksi database, bukan mewarisi satu sama lain), Dependency Injection (dependensi diberikan lewat constructor, bukan dibuat sendiri oleh class yang memakainya), serta Encapsulation melalui getter dan setter yang divalidasi. Dengan pola ini, Repository dapat diuji atau diganti implementasinya tanpa mengubah kode Controller.

2. Tujuan

1. Membuat class Database yang mengelola koneksi ke database menggunakan PDO.
2. Membuat class MahasiswaRepository yang menerima objek Database melalui constructor (constructor injection) untuk operasi CRUD data mahasiswa.
3. Membuat class MahasiswaController yang menerima objek MahasiswaRepository melalui constructor, sehingga Controller tidak lagi membuat koneksi database secara langsung.
4. Mengubah seluruh atribut pada class Mahasiswa menjadi private, lalu membuat getter dan setter untuk mengakses serta mengubah data.
5. Menambahkan validasi sederhana pada setter: nama mahasiswa tidak boleh kosong, dan NIM harus berupa angka.
6. Menjalankan aplikasi untuk memastikan proses tambah, ubah, tampil, dan hapus data tetap berjalan setelah refactoring.

3. Dasar Teori Singkat

Object Composition. Lebih baik sebuah class “memiliki” (has-a) object lain daripada mewarisi (is-a) terlalu dalam. Pada acara ini, `MahasiswaController` memiliki `MahasiswaRepository`, dan `MahasiswaRepository` memiliki koneksi PDO. Tidak ada relasi pewarisan (extends) di antara ketiganya, sehingga masing-masing class dapat diubah tanpa memengaruhi yang lain.

Dependency Injection (Constructor Injection). Alih-alih sebuah class membuat sendiri dependensinya (misalnya memanggil `new PDO(...)` langsung di dalam method), dependensi tersebut “suntikkan” dari luar melalui parameter constructor. `MahasiswaRepository` menerima PDO lewat constructor-nya, dan `MahasiswaController` menerima `MahasiswaRepository` dengan cara yang sama.

Getter dan Setter dengan Validasi. Seluruh atribut pada Entity Mahasiswa dijadikan `private`, sehingga hanya dapat dibaca lewat getter dan diubah lewat setter. Setter adalah tempat yang tepat untuk menaruh validasi (misalnya NIM harus angka), karena validasi berjalan otomatis setiap kali data diubah, dari jalur manapun perubahan itu berasal.

4. Alat dan Bahan

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon
- PHP
- HTML, CSS, Bootstrap
- Modul Praktikum (BKPM Acara 9)

HASIL PRAKTIK

1. Class Database (Singleton, dari Acara 8)

Class Database yang dibuat pada Acara 8 dipakai kembali tanpa perubahan, sesuai langkah kerja butir 1. Class ini menyediakan satu koneksi PDO yang sama untuk seluruh Repository.

```
app > Core > Database.php
1  <?php
2
3  class Database
4  {
5      private static ?PDO $instance = null;
6
7      public static function getInstance(): PDO
8      {
9          if (self::$instance === null) {
10             $config = require __DIR__ . '/../config/database.php';
11             $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['charset']}";
12
13             self::$instance = new PDO($dsn, $config['username'], $config['password'], [
14                 PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
15                 PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
16                 PDO::ATTR_EMULATE_PREPARES  => false,
17             ]);
18         }
19
20         return self::$instance;
21     }
22 }
```

2. Entity Mahasiswa: Atribut Private, Getter, dan Setter

Seluruh atribut pada class Mahasiswa diubah menjadi private. Setter setNim() dan setName() memvalidasi datanya (langkah kerja butir 4 dan 5); apabila tidak valid, setter melempar InvalidArgumentException.

```
app > Models > Mahasiswa.php
1  <?php
2
3  class Mahasiswa
4  {
5      private ?int $id = null;
6      private string $nim;
7      private string $nama;
8      private int $prodiId;
9      private int $angkatan;
10     private string $status;
11
12     private ?string $prodiNama = null;
13
14     public function __construct(string $nim, string $nama, int $prodiId, int $angkatan, string $status = 'aktif')
15     {
16         $this->setNim($nim);
17         $this->setName($nama);
18         $this->prodiId = $prodiId;
19         $this->angkatan = $angkatan;
20         $this->setStatus($status);
21     }
22
23     // GETTER
24     public function getId(): ?int { return $this->id; }
25     public function getNim(): string { return $this->nim; }
26     public function getName(): string { return $this->nama; }
27     public function getProdiId(): int { return $this->prodiId; }
28     public function getAngkatan(): int { return $this->angkatan; }
29     public function getStatus(): string { return $this->status; }
30     public function getStatusLabel(): string { return ucfirst($this->status); }
31     public function getProdiNama(): string { return $this->prodiNama ?? '-'; }
32
33     // SETTER + VALIDASI
34     public function setId(int $id): void
35     {
36         $this->id = $id;
37     }
38
39     public function setNim(string $nim): void
40     {
41         if (!ctype_digit($nim)) {
42             throw new InvalidArgumentOutOfRangeException('NIM harus berupa angka.');
```

3. MahasiswaRepository: Constructor Injection

Repository menerima PDO melalui constructor (bukan membuatnya sendiri), lalu menyediakan seluruh operasi CRUD. Setiap baris hasil query diubah (di-hydrate) menjadi object Mahasiswa, bukan array asosiatif mentah, sehingga Controller dan View selalu bekerja dengan object.

```
app > Repositories > MahasiswaRepository.php
1  <?php
2
3  class MahasiswaRepository
4  {
5      private PDO $db;
6
7      public function __construct(PDO $db)
8      {
9          $this->db = $db;
10     }
11
12     // Daftar mahasiswa + nama prodi (JOIN)
13     public function allWithProdi(?string $keyword = null): array
14     {
15         $sql = "SELECT m.*, p.nama AS prodi_nama
16                 FROM mahasiswa m
17                 JOIN prodi p ON m.prodi_id = p.id";
18
19         $params = [];
20         if ($keyword !== null && $keyword !== '') {
21             $sql .= " WHERE m.nama LIKE :kw_nama OR m.nim LIKE :kw_nim";
22             $params['kw_nama'] = '%' . $keyword . '%';
23             $params['kw_nim'] = '%' . $keyword . '%';
24         }
25         $sql .= " ORDER BY m.nim";
26
27         $stmt = $this->db->prepare($sql);
28         $stmt->execute($params);
29
30         return array_map([$this, 'hydrate'], $stmt->fetchAll());
31     }
32
33     // Mengubah satu baris hasil query menjadi object Mahasiswa (Entity).
34     private function hydrate(array $row): Mahasiswa
35     {
36         $mhs = new Mahasiswa(
37             $row['nim'],
38             $row['nama'],
39             (int) $row['prodi_id'],
40             (int) $row['angkatan'],
41             $row['status']
42         );
43         $mhs->setId((int) $row['id']);
44
45         if (isset($row['prodi_nama'])) {
46             $mhs->setProdiNama($row['prodi_nama']);
47         }
48
49         return $mhs;
50     }
51
52     // Mengubah object Mahasiswa menjadi array parameter untuk PDO.
53     private function toParams(Mahasiswa $mhs): array
54     {
55         return [
56             'nim' => $mhs->getNim(),
57             'nama' => $mhs->getNama(),
58             'prodi_id' => $mhs->getProdiId(),
59             'angkatan' => $mhs->getAngkatan(),
60             'status' => $mhs->getStatus(),
61         ];
62     }
63 }
```

4. MahasiswaController: Menerima Repository Lewat Constructor

Controller tidak lagi memanggil Database::getInstance() untuk dirinya sendiri secara berulang di setiap method. Ia hanya membuat satu MahasiswaRepository sekali, di dalam constructor, lalu memakainya di semua method (index, detail, store, update, destroy).

```
app > Controllers > MahasiswaController.php
1  <?php
2
3  class MahasiswaController extends Controller
4  {
5      private MahasiswaRepository $repo;
6
7      public function __construct()
8      {
9          $this->repo = new MahasiswaRepository(Database::getInstance());
10     }
11
12     public function index(): void
13     {
14         $keyword      = trim($_GET['q'] ?? '');
15         $mahasiswaList = $this->repo->allWithProdi($keyword !== '' ? $keyword : null);
16         $prodiList     = Prodi::all();
17         $flash         = flash_get();
18
19         $this->view('mahasiswa/index', compact('mahasiswaList', 'prodiList', 'keyword', 'flash'));
20     }
```

```
46     public function store(): void
47     {
48         try {
49             $mhs = $this->buildEntityFromRequest();
50             $this->repo->create($mhs);
51             flash_set('success', 'Data mahasiswa berhasil ditambahkan.');
```

```
102     private function buildEntityFromRequest(): Mahasiswa
103     {
104         $nim      = trim($_POST['nim'] ?? '');
105         $nama     = trim($_POST['nama'] ?? '');
106         $prodiId  = (int) ($_POST['prodi_id'] ?? 0);
107         $angkatan = (int) ($_POST['angkatan'] ?? date('Y'));
108         $status   = $_POST['status'] ?? 'aktif';
109
110         $mhs = new Mahasiswa($nim, $nama, $prodiId, $angkatan, $status);
111         $mhs->setProdiId($prodiId);
112
113         return $mhs;
114     }
115 }
```

View juga disesuaikan: karena Repository mengembalikan object, bukan array, tampilan memanggil method getter, misalnya \$mhs->getNim() dan \$mhs->getNama(), menggantikan akses array \$mhs['nim'] yang dipakai pada Acara 7-8.

5. Hasil Pengujian

a) Daftar mahasiswa: tetap tampil dengan benar (JOIN, pencarian, dan flash message tidak berubah) walau sumber datanya kini berupa object, bukan array.

Data Mahasiswa
Daftar mahasiswa yang terdaftar

Selamat datang, **admin**

Daftar Mahasiswa 4 data

Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
2401001	Budi Santoso	Teknik Informatika	2024	Aktif	Detail Edit Delete
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail Edit Delete
2401003	Asti	Teknik Informatika	2026	Aktif	Detail Edit Delete
2402001	Citra Lestari	Sistem Informasi	2026	Aktif	Detail Edit Delete

Logout

b) Validasi setter: percobaan menyimpan NIM “ABCDE” (bukan angka) ditolak oleh setNim() sebelum sempat disimpan ke database.

Tambah Mahasiswa
Isi form berikut untuk menambah data mahasiswa baru

⚠ NIM harus berupa angka.

NIM
ABCDE

Nama Lengkap
Yuli

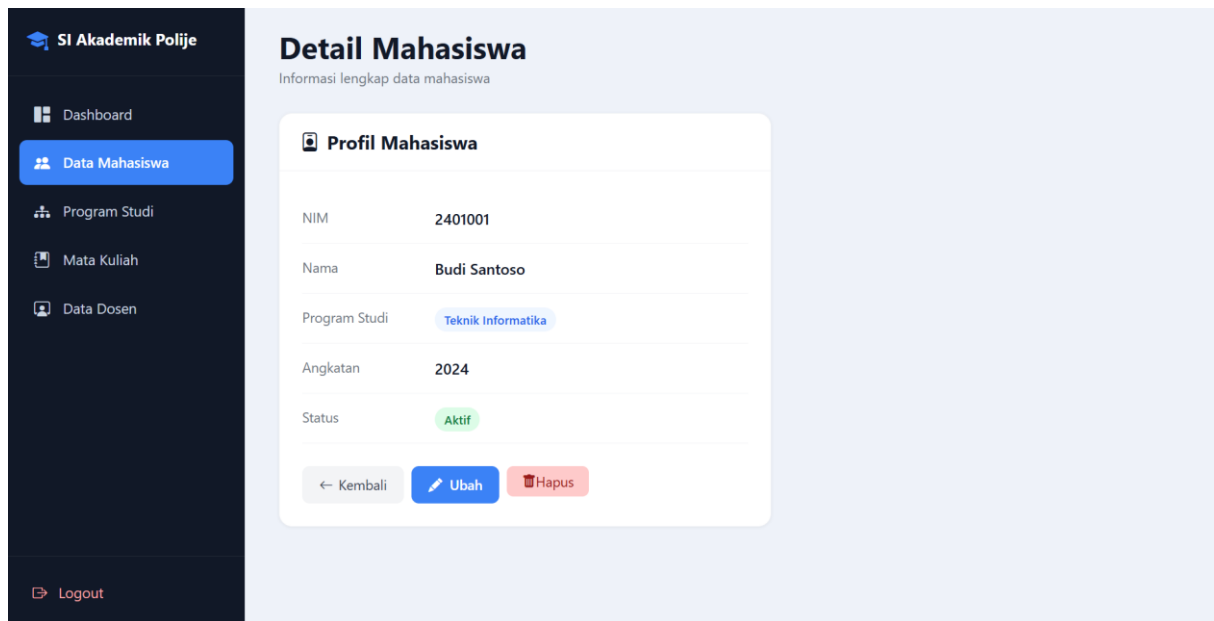
Program Studi
Teknik Informatika

Angkatan
2026

Status
Aktif

Logout

c) Halaman detail tetap menampilkan data lengkap, kini seluruhnya lewat getter Entity.



6. Analisis dan Pembahasan

Pengujian menunjukkan bahwa seluruh alur tambah, ubah, tampil, hapus, dan pencarian pada Acara 7-8 tetap berjalan normal setelah refactoring (langkah kerja butir 6), karena perubahan hanya terjadi pada bagaimana data mengalir secara internal (dari array menjadi object), bukan pada perilaku aplikasi yang terlihat pengguna.

Manfaat Dependency Injection terlihat jelas saat pengujian: MahasiswaController tidak pernah memanggil PDO atau Database::getInstance() di dalam method-method CRUD-nya, karena satu MahasiswaRepository sudah cukup dibuat sekali di constructor dan dipakai berulang. Apabila suatu saat Repository perlu diganti implementasinya (misalnya untuk pengujian otomatis dengan data palsu), cukup mengganti object yang disuntikkan ke constructor, tanpa menyentuh kode Controller sama sekali.

Validasi pada setter juga terbukti konsisten dari jalur manapun: baik saat object Mahasiswa dibuat dari input form (tambah data) maupun ketika di-hydrate dari baris hasil query database, keduanya sama-sama lewat constructor yang sama, sehingga aturan yang sama (NIM harus angka, nama tidak boleh kosong, email harus valid) berlaku di kedua arah, bukan hanya saat data masuk dari form.

KESIMPULAN

1. Object Composition diterapkan dengan menjadikan MahasiswaController memiliki MahasiswaRepository, dan MahasiswaRepository memiliki koneksi PDO, tanpa relasi pewarisan di antara ketiganya.
2. Dependency Injection (constructor injection) membuat MahasiswaRepository dan MahasiswaController menerima dependensinya dari luar, sehingga tidak ada lagi pembuatan koneksi database secara langsung di dalam Controller.
3. Seluruh atribut Entity Mahasiswa dijadikan private dan hanya dapat diakses lewat getter, serta diubah lewat setter yang memvalidasi datanya.
4. Validasi sederhana pada setter (NIM harus angka, nama tidak boleh kosong, serta validasi tambahan format email) berhasil mencegah data tidak valid tersimpan ke database, baik lewat form tambah maupun ubah data.
5. Setelah refactoring, seluruh proses tambah, ubah, tampil, hapus, dan pencarian data mahasiswa tetap berjalan seperti sebelumnya, membuktikan bahwa perubahan struktur kode (menuju pola yang lebih rapi dan mudah diuji) tidak harus mengubah perilaku aplikasi yang terlihat pengguna.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.